AWS 기반의 CI/CD 파이프라인을 통해 자동화된 배포

201916136
진덕종

목차

01 프로젝트 서론

02 AWS ECR

03 AWS VPC

04 AWS RDS

05 AWS SECRET

06 AWS ElastiCache

07 AWS ECS

08 CI/CD

CI/CD

PART 1.

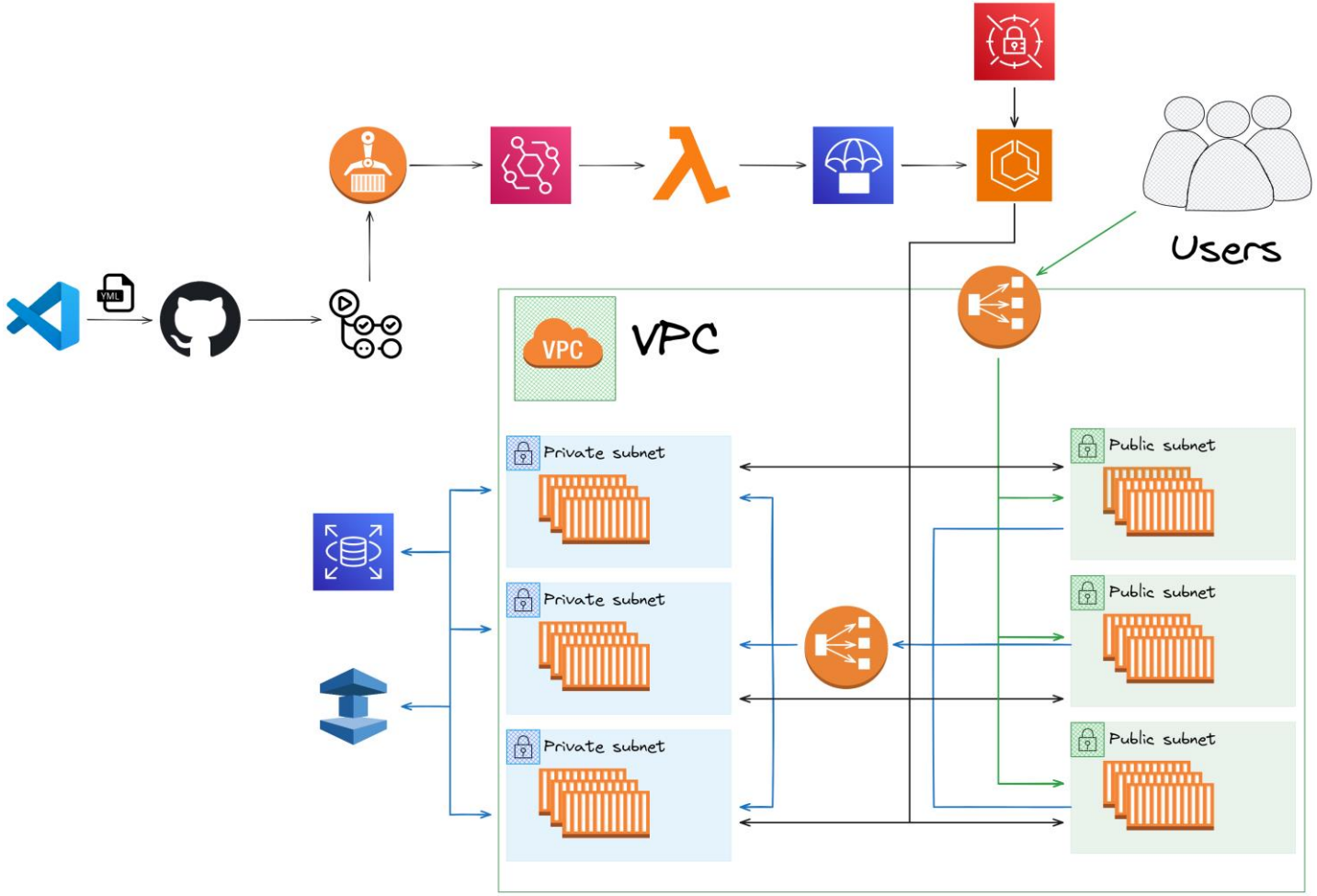
프로젝트 서론

Part 1

프로젝트 서론

- 원래 기존에 하던 Full Stack 프로젝트가 있음.
- 이 프로젝트는 간단한 쇼핑몰을 만들던 프로젝트로 Next.js와 Spring boot를 이용하고 있음.
- 이제까지 작동 할 때 Docker를 이용해서 띄우고 있었음. 이후 웹 서비스 설계라는 과목을 들으면서 Jcloud 인스턴스에 한 서비스로 DB, Redis, Backend 가 모두 묶여 올라가는 것이 조금 아쉬웠음.
- 이번 프로젝트를 진행하면서 모든 저장소와 Application을 분리하고, 각 Application 마다 컨테이너를 띄우며 CI/CD 작업을 도입해보고자 마음을 먹음.

프로젝트 서론


Users
VPC
VPC
Private subnet
Private subnet
Private subnet
Public subnet
Public subnet
Public subnet

Part 1

프로젝트 서론

ECR과 Git Action

Docker Image

Github action을 이용해 코드가 레포지토리에 올라오면 ECR로 이미지 배포

RDS 생성

PostgreSQL 생성

ElastiCache

Redis 생성

CI/CD

Green-Blue

EventBridge를 이용해 트리거를 삼아 Lambda 함수를 실행시켜 CodeDeploy를 이용

VPC 설정

Backend와 Frontend 분리

Frontend Application은 모든 Http요청에 응답을 하지만, Backend Application은 오직 Frontend Application에서만 오는 Http 요청을 허용

SECRET MANAGER

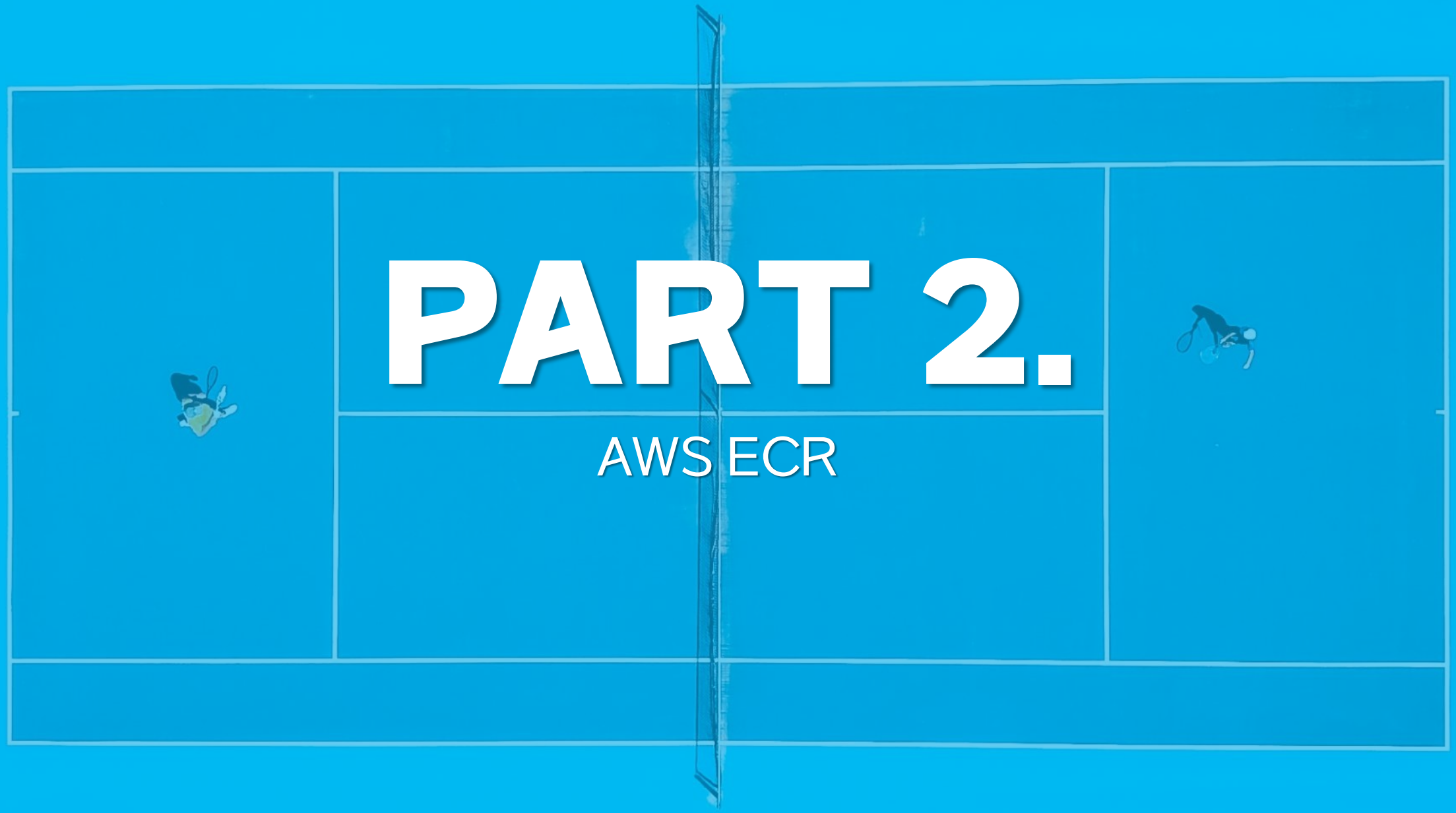
환경변수의 관리

Task를 실행시키면서 자동으로 환경변수를 주입하기 위해 사용

ECS

Task 정의와 실행

각 도커 이미지를 실행하는 Task를 정의하고, 클러스터에 컨테이너 생성



PART 2.

AWS ECR

AWS ECR

STEP 1		STEP 2		STEP 3
Github에 특정 브랜치에 코드가 업로드됨	>>	미리 정의한 Depoly.yml 파일에 의해 Actions 시작	>>	도커를 이용해 이미지 빌드 후 AWS 자격인증을 한 뒤 ECR에 업로드 됨

Part 2

AWS ECR

```yaml
jobs:
  build-and-push:
    runs-on: ubuntu-latest
    environment: ${{ github.ref == 'refs/heads/main' && 'production' || 'test' }} # 메인 브랜치에 푸시될 때 프로덕션 환경으로 배
    strategy:
      matrix:
        service: [backend, frontend]
    steps:
      - name: 코드 체크아웃
        uses: actions/checkout@v2

      - name: AWS 자격 증명 구성
        uses: aws-actions/configure-aws-credentials@v1
        with:
          aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: ${{ env.AWS_REGION }}

      - name: Amazon ECR 로그인
        id: login-ecr
        uses: aws-actions/amazon-ecr-login@v1


      - name: 프로젝트 환경 변수 설정
        run: |
          touch ./frontend/.env
          echo "${{ secrets.FRONTEND_SECRET }}" >> ./frontend/.env
          touch ./backend/.env
          echo "${{ secrets.BACKEND_SECRET }}" >> ./backend/.env

      - name: 이미지 빌드 및 푸시
        env:
          ECR_REGISTRY: ${{ steps.login-ecr.outputs.registry }}
          IMAGE_TAG: "${{ github.sha }}-${{ matrix.service }}"
        run: |
          echo "📦 ${{ matrix.service }} 이미지 빌드 및 푸시 시작"
          docker build -t $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG -f ./${{ matrix.service }}/Dockerfile ./${{ matrix.service }}
          docker push $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG
          echo "🚀 ${{ matrix.service }} 이미지 빌드 및 푸시 완료"
```

Part 2

AWS ECR

프라이빗 리포지토리 생성

일반 설정

리포지토리 이름

간결한 이름을 입력합니다. 리포지토리 이름은 유사한 리포지토리를 그룹화하는 데 권장되는 네임스페이스를 지원합니다.

11698[redacted]

19로 최대 문자 수 256(최소 2)을(를) 초과했습니다. 이름은 문자로 시작해야 하며 소문자, 숫자 및 특수 문자(._-/)만 포함할 수 있습니다.

이미지 태그 변경 가능성 | 정보

사용할 태그 변경 가능성 설정을 지정합니다. 리포지토리에 대해 태그 변경 불가능을 설정하면 태그를 덮어쓸 수 없습니다.

- (●) Mutable
  이미지 태그를 덮어쓸 수 있습니다.
- ( ) Immutable
  이미지 태그를 덮어쓸 수 없습니다.

암호화 설정

⚠ 리포지토리 생성 후에는 리포지토리의 암호화 설정을 변경할 수 없습니다.

암호화 구성 | 정보

기본적으로 리포지토리는 업계 표준 Advanced Encryption Standard (AES) 암호화를 사용합니다. 선택적으로 AWS Key Management Service (KMS)에 저장된 키를 사용하여 리포지토리의 이미지를 암호화하도록 선택할 수 있습니다.

- ( ) AES-256
  업계 표준 Advanced Encryption Standard (AES) 암호화
- (●) AWS KMS
  AWS Key Management Service (KMS)

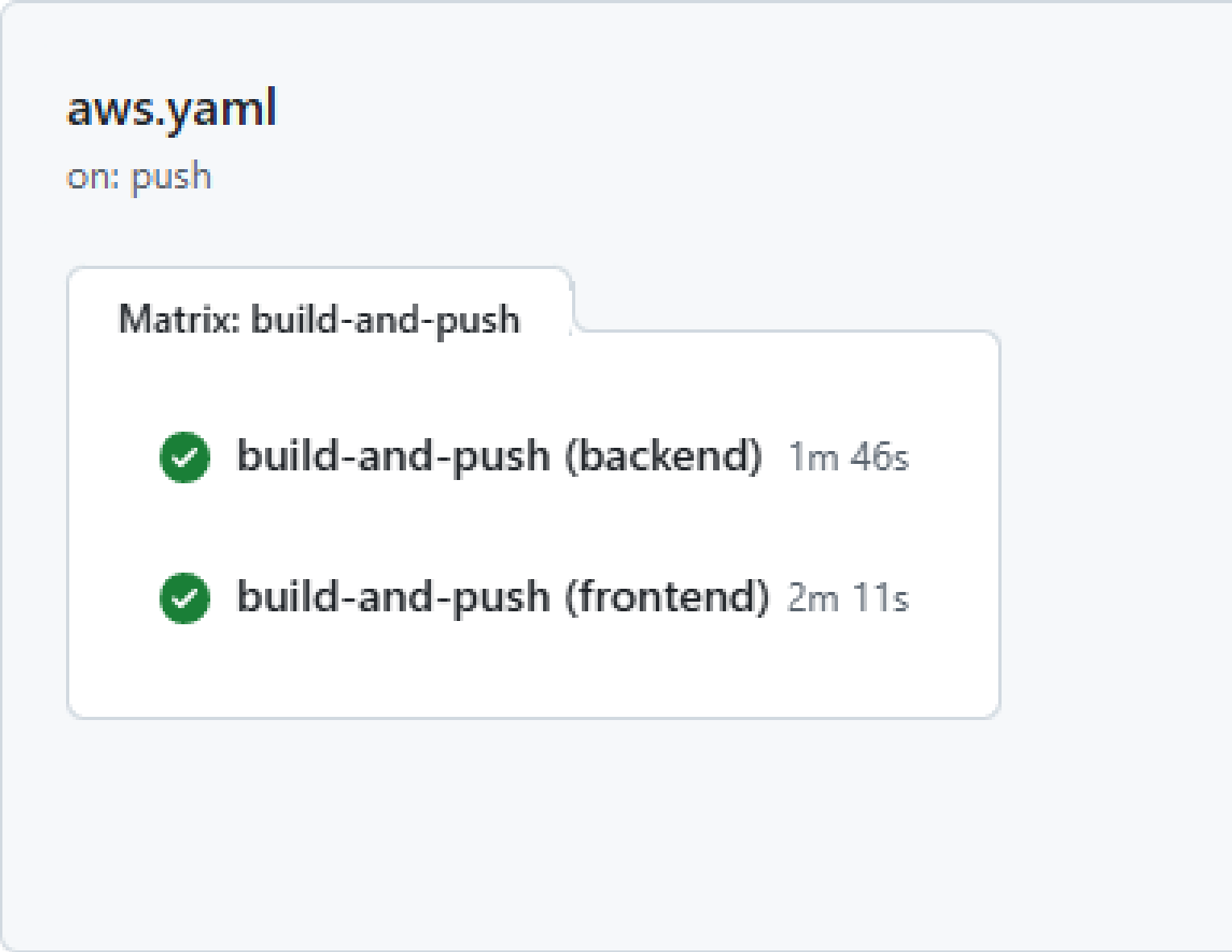
AWS KMS 암호화 설정

Amazon ECR이(가) 관리하는 기본 AWS 관리형 키를 사용할지 아니면 AWS KMS 서비스에 저장된 자체 키를 지정할지 선택합니다. 자세히 알아보기

- (●) AWS 관리형 키
  AWS 관리형 키를 사용합니다.
- ( ) 고객 관리형 키
  AWS KMS 서비스에 저장되는 고객 관리형 키를 생성하거나 기존 고객 관리형 키를 선택합니다.

Part 2

AWS ECR



Part 2

AWS ECR

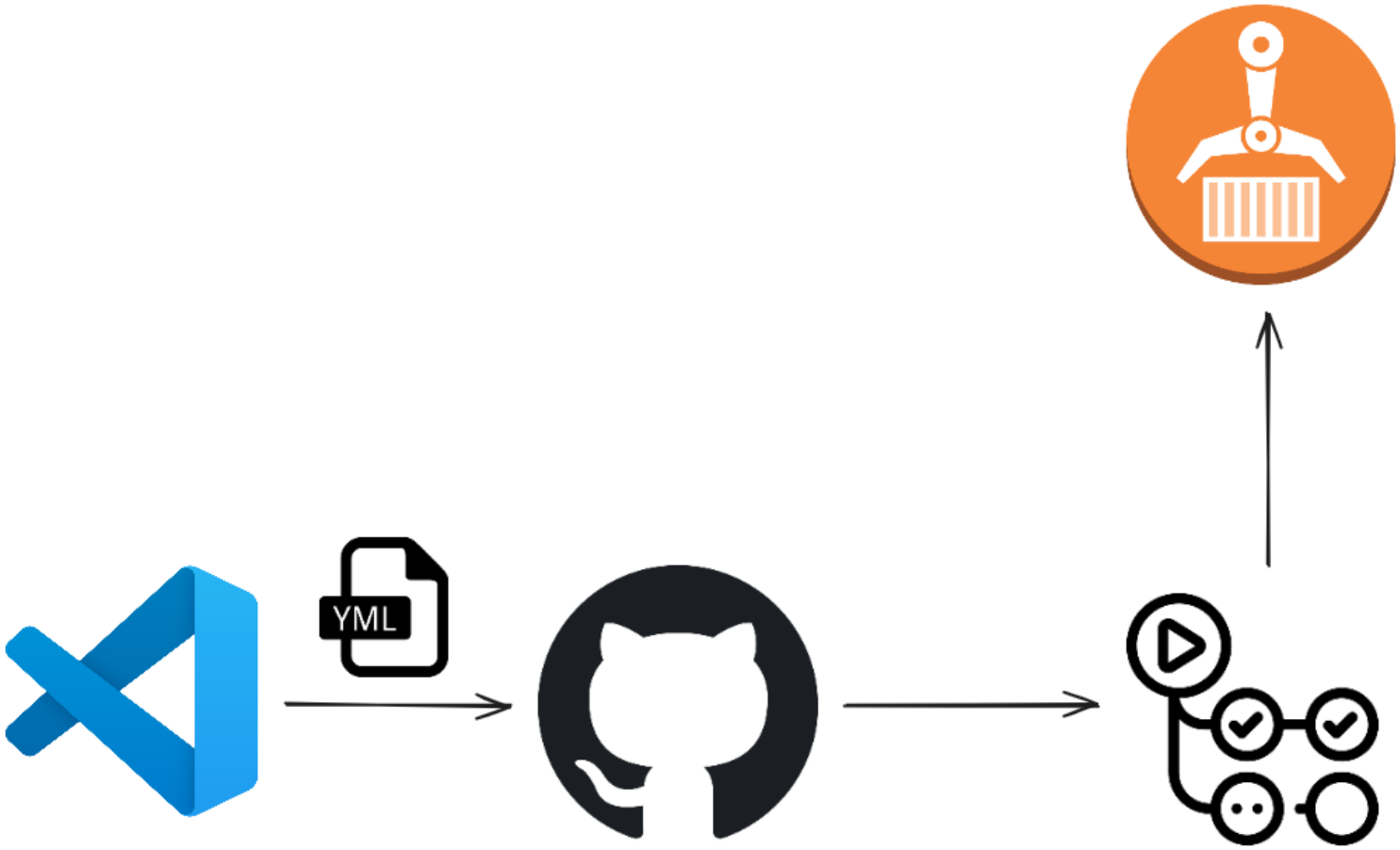
아티팩트 검색

1

이미지 태그	아티팩트 유형	푸시 위치	크기(MB)	이미지 URI	다이제스트
311bc30cb1a7f5ef7c4f3d7a7c4e2f3b2b40aa03-frontend	Image	2024년 12월 18일, 22:51:42 (UTC+09)	244.23	URI 복사	sha256:e6f9af81f5c3ccc7...
311bc30cb1a7f5ef7c4f3d7a7c4e2f3b2b40aa03-backend	Image	2024년 12월 18일, 22:49:42 (UTC+09)	129.41	URI 복사	sha256:bfb99e99ca832ea...
c99834fd399e0a3da9279333ae6184f5d31867fe	Image	2024년 12월 18일, 22:42:01 (UTC+09)	244.23	URI 복사	sha256:e0148043a6c022...
-	Image	2024년 12월 18일, 22:40:02 (UTC+09)	129.41	URI 복사	sha256:27a2454baa1408...

Part 2

AWS ECR



PART 3.

AWS VPC

Part 3

AWS VPC

Spring boot

Private VPC에 존재.
Public VPC의 인바운드는 오직
Next.js의 보안 그룹만 허용

RDS, Redis

오직 Private에 있는 Spring boot VPC만 접근을 허용함

Next.js

Public VPC에 존재.
모든 HTTP 인바운드를 허용함.

Part 3

AWS VPC

VPC 생성 정보

VPC는 AWS 클라우드의 격리된 부분으로서, Amazon EC2 인스턴스와 같은 AWS 객체로 채워집니다. 리소스를 마우스로 가리키면 관련 리소스가 강조 표시됩니다.

VPC 설정

생성할 리소스 정보

VPC 리소스 또는 VPC 및 기타 네트워킹 리소스만 생성합니다.

- ○ VPC만
- ◉ VPC 등

이름 태그 자동 생성 정보

이름 태그의 값을 입력합니다. 이 값은 VPC의 모든 리소스에 대한 이름 태그를 자동으로 생성하는 데 사용됩니다.

☑ 자동 생성

jbnu-shopping

IPv4 CIDR 블록 정보

CIDR 표기법을 사용하여 VPC의 시작 IP와 크기를 결정합니다.

10.0.0.0/16 65,536 IPs

CIDR 블록 크기는 /16에서 /28 사이여야 합니다.

IPv6 CIDR 블록 정보

- ◉ IPv6 CIDR 블록 없음
- ○ Amazon 제공 IPv6 CIDR 블록

테넌시 정보

기본값

미리 보기

VPC 세부 정보 표시

AWS 가상 네트워크

- jbnu-shopping-vpc

서브넷(6개)

이 VPC 내의 서브넷

ap-northeast-2a
- A jbnu-shopping-subnet-public1-ap-
- A jbnu-shopping-subnet-private1-ap-

ap-northeast-2b
- B jbnu-shopping-subnet-public2-ap-
- B jbnu-shopping-subnet-private2-ap-

ap-northeast-2c
- C jbnu-shopping-subnet-public3-ap-
- C jbnu-shopping-subnet-private3-ap-

라우팅 테이블(4개)

네트워크 트래픽을 리소스로 라우팅

- jbnu-shopping-rtb-public
- jbnu-shopping-rtb-private1-ap-
- jbnu-shopping-rtb-private2-ap-
- jbnu-shopping-rtb-private3-ap-

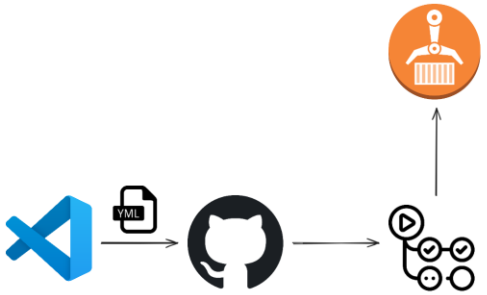
네트워크 연결(3개)

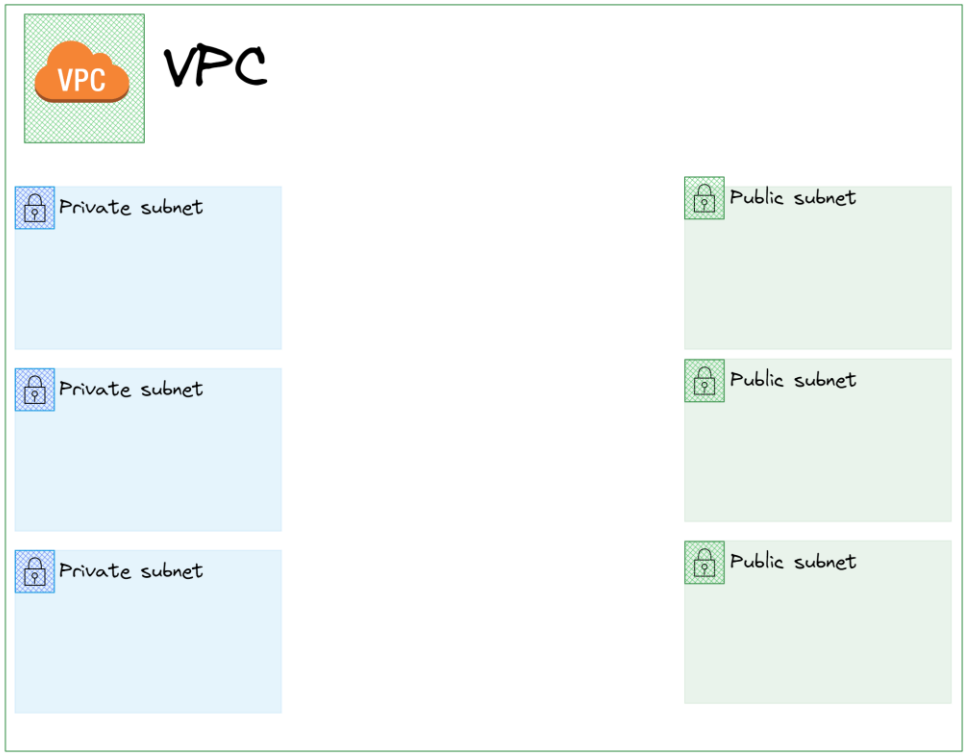
다른 네트워크에 연결

- jbnu-shopping-igw
- jbnu-shopping-nat-public1-ap-
- jbnu-shopping-vpce-s3

Part 3

AWS VPC





PART 4.

AWS RDS

AWS RDS

001 >> PostgreSQL

해당 Spring Boot Project는 PostgreSQL을 사용합니다

002 >> 인바운드 규칙은 Spring boot 보안그룹

오직 Spring Boot의 보안그룹만의 요청을 받습니다

AWS RDS

연결

네트워크 유형 정보
듀얼 스택 모드를 사용하려면 IPv6 CIDR 블록을 지정한 VPC의 서브넷과 연결해야 합니다.

- (선택됨) **IPv4**
  리소스는 IPv4 주소 지정 프로토콜을 통해서만 통신할 수 있습니다.
- **듀얼 스택 모드**
  리소스는 IPv4, IPv6 또는 둘 모두를 통해 통신할 수 있습니다.

DB 서브넷 그룹

default-vpc-04b952b63e13a4e0b

보안 그룹
이 DB 인스턴스와 연결할 DB 보안 그룹 목록입니다.

보안 그룹 선택

default ×

인증 기관 정보
서버 인증서를 사용하면 Amazon 데이터베이스에 대한 연결이 이루어지고 있는지 검증하여 추가 보안 계층을 제공합니다. 프로비저닝하는 모든 데이터베이스에 자동으로 설치되는 서버 인증서를 확인하여 이를 수행합니다.

rds-ca-rsa2048-g1 (기본값)
만료: May 21, 2061

▼ 추가 구성

퍼블릭 액세스

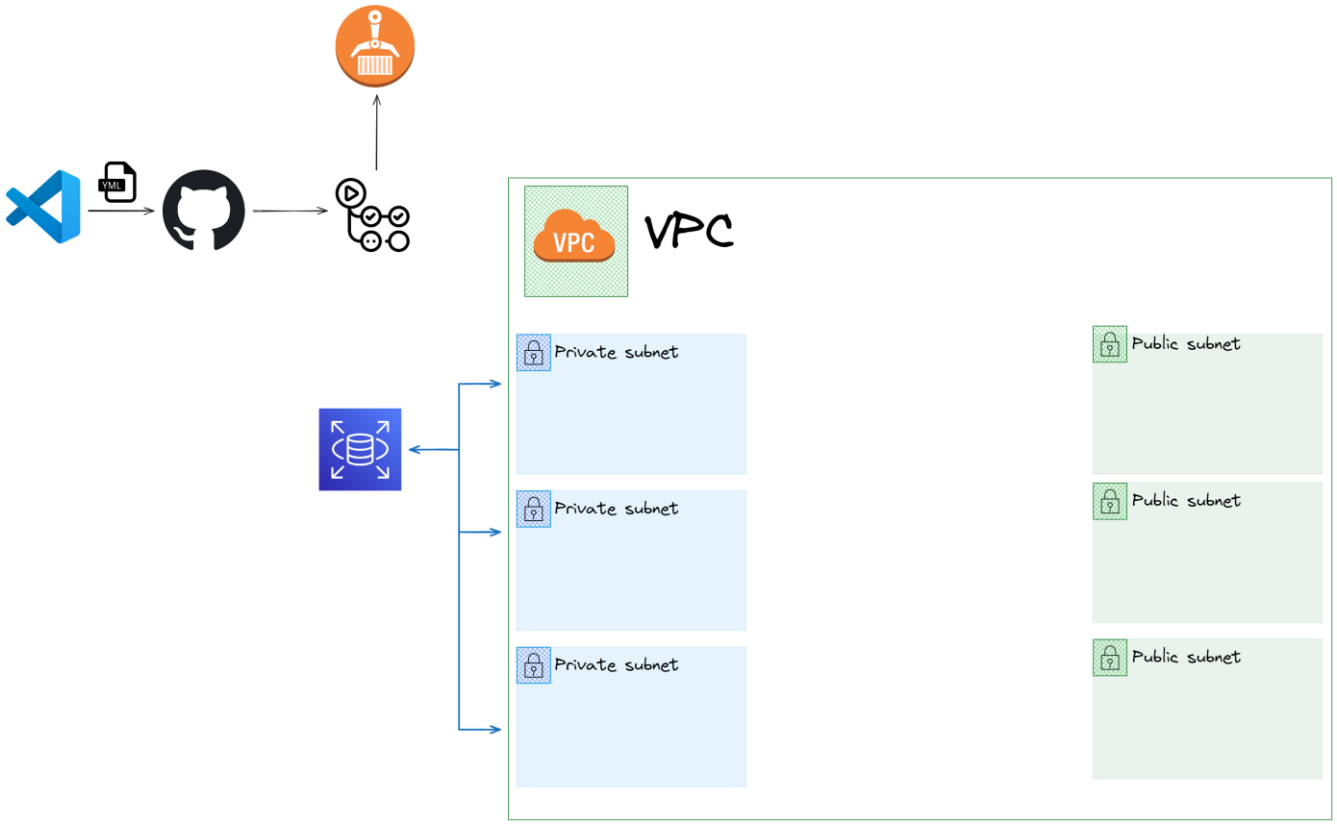
- **퍼블릭 액세스 가능**
  RDS는 데이터베이스에 퍼블릭 IP 주소를 할당합니다. VPC 외부의 Amazon EC2 인스턴스 및 다른 리소스가 데이터베이스에 연결할 수 있습니다. VPC 내부의 리소스도 데이터베이스에 연결할 수 있습니다. 데이터베이스에 연결할 수 있는 리소스를 지정하는 VPC 보안 그룹을 하나 이상 선택합니다.
- (선택됨) **퍼블릭 액세스 불가능**
  DB 인스턴스에 할당된 IP 주소가 없습니다. VPC 외부의 EC2 인스턴스 및 디바이스는 연결할 수 없습니다.

데이터베이스 포트
DB 인스턴스가 애플리케이션 연결에 사용할 TCP/IP 포트를 지정합니다. 애플리케이션 연결 문자열에서 포트 번호를 지정해야 합니다. DB 보안 그룹과 방화벽은 이 포트에 대한 연결을 허용해야 합니다.자세히 알아보기

5432

Part 4

AWS RDS



PART 5.

AWS SECRET

AWS SECRET

001 » 환경변수를 클라우드에서 관리

Spring boot의 경우 Bootstracp.yml를 통해 환경변수를 AWS의 SECRETS에서 받아옵니다.

보안 암호 이름	설명	마지막 검색 날짜(UTC)
/secrets[redacted]	jbnu-shopping의 Backend 환경 변수	-
rds[redacted]	The secret associated with the primary RDS DB instance: arn:aws:rds:ap-northeast-2:116981803095:db:jbnu-shopping-database-1	2024년 12월 19일

Part 5

AWS SECRET-TEST

```yaml
spring:
  config:
    import:
      - "aws-secretsmanager:/secrets/[redacted] backend 키 이름
      - "aws-secretsmanager:arn:aws:[redacted]

cloud:
  aws:
    region:
      static: ap-northeast-2
```

```yaml
spring:
  datasource:        You, 11시간 전 • feature: aws 템플릿 사용 추가
    url: "jdbc-secretsmanager:postgresql://${RDS_ENDPOINT}/${DB_NAME}?secretArn=arn:aws:s
    driver-class-name: com.amazonaws.secretsmanager.jdbc.AWSSecretsManagerDriver

  data:
    redis:
      host: ${ELASTICACHE_HOST}
      port: ${ELASTICACHE_PORT}
      password: ${ELASTICACHE_PASSWORD}
```

Part 5

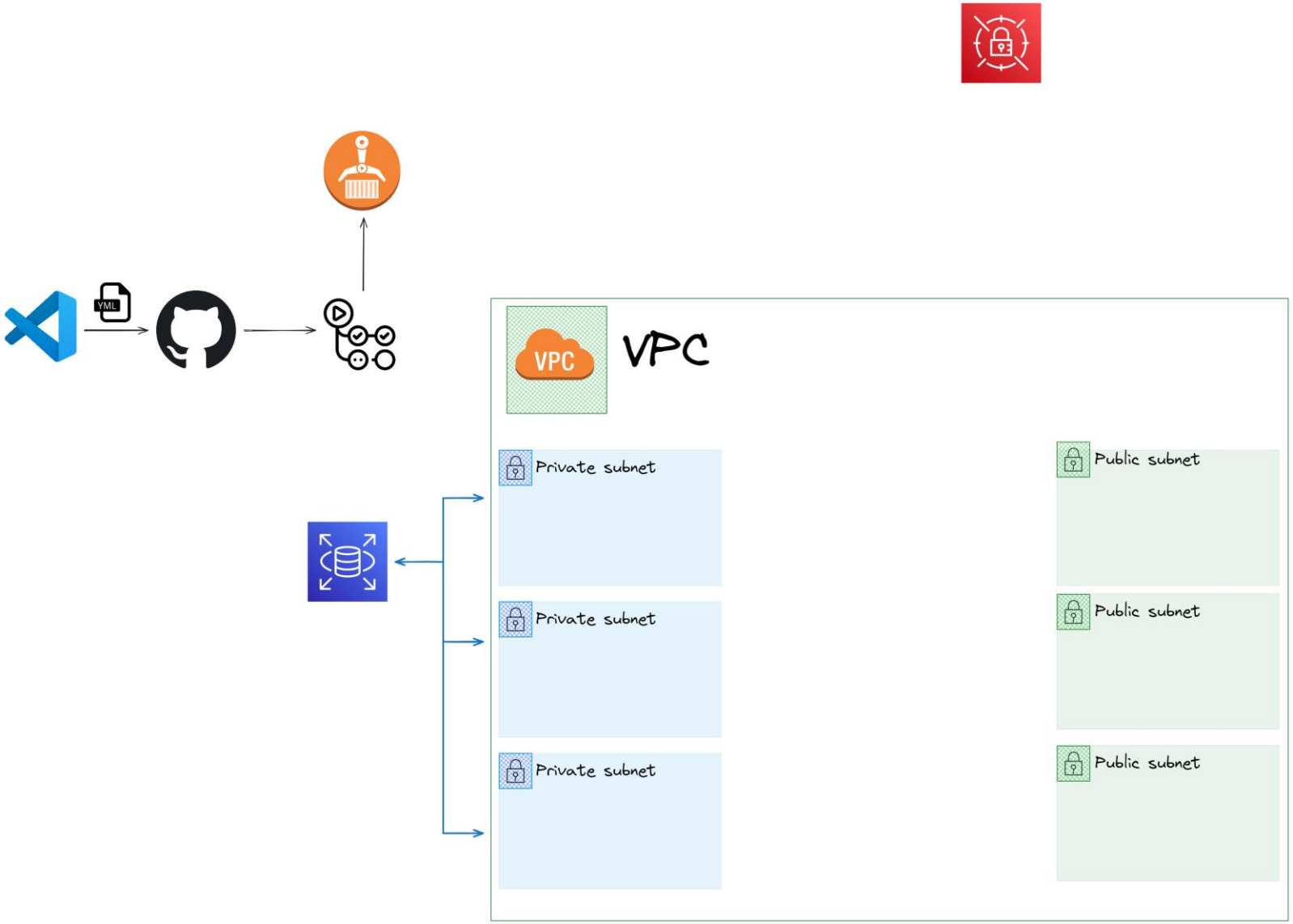
AWS SECRET-TEST

```java
@Value("${RDS_ENDPOINT}")
private String url;

@Value("${username}")
private String username;

@Value("${password}")
private String password;

Run | Debug
public static void main(String[] args) {
    SpringApplication.run(primarySource:BackendApplication.class, args);
}

@PostConstruct
public void init() {        You, 3분 전 • Uncommitted changes
    log.info(format:"RDS_ENDPOINT: {}", url);
    log.info(format:"username: {}", username);
    log.info(format:"password: {}", password);
}
```


```
}
{
  "@timestamp" : "2024-12-19T23:30:23.910+09:00",
  "message" : "RDS_ENDPOINT: [redacted]st-2.rds.amazonaws.com",
  "logger" : "com.cottongallery.backend.BackendApplication",
  "thread" : "restartedMain",
  "level" : "INFO"
}
{
  "@timestamp" : "2024-12-19T23:30:23.910+09:00",
  "message" : "username: postgres",
  "logger" : "com.cottongallery.backend.BackendApplication",
  "thread" : "restartedMain",
  "level" : "INFO"
}
{
  "@timestamp" : "2024-12-19T23:30:23.911+09:00",
  "message" : "password: [redacted]",
  "logger" : "com.cottongallery.backend.BackendApplication",
  "thread" : "restartedMain",
  "level" : "INFO"
}
```

Part 5

AWS SECRET-TEST



PART 6.

AWS ElastiCache

AWS ElastiCache

001 >> Redis

현재 Spring boot는 Redis를 필요로 합니다.
Redis에다가 Refresh Token을 저장하여 만료시간을 지정하고, 요청이 들어오면 Redis에서 확인합니다.
Admin 계정을 정적으로 지정하는 것이 아닌, 계속해서 생성하고 제거되기에 Redis에 저장합니다.

002 >> 보안그룹

Elasticache의 보안 그룹은 Private VPC 내부 Spring boot 컨테이너만 접근하게 합니다.

AWS ElastiCache

검토 및 생성 정보

1단계: 클러스터 설정

편집

클러스터 정보
다음 옵션을 사용하여 클러스터를 구성합니다.

이름	설명
redis-1	-

위치
클러스터를 AWS 클라우드에서 호스팅할지, 온프레미스에서 호스팅할지 선택합니다.

위치	클러스터 모드
aws-cloud	비활성화됨

클러스터 설정
다음 옵션을 사용하여 클러스터를 구성합니다.

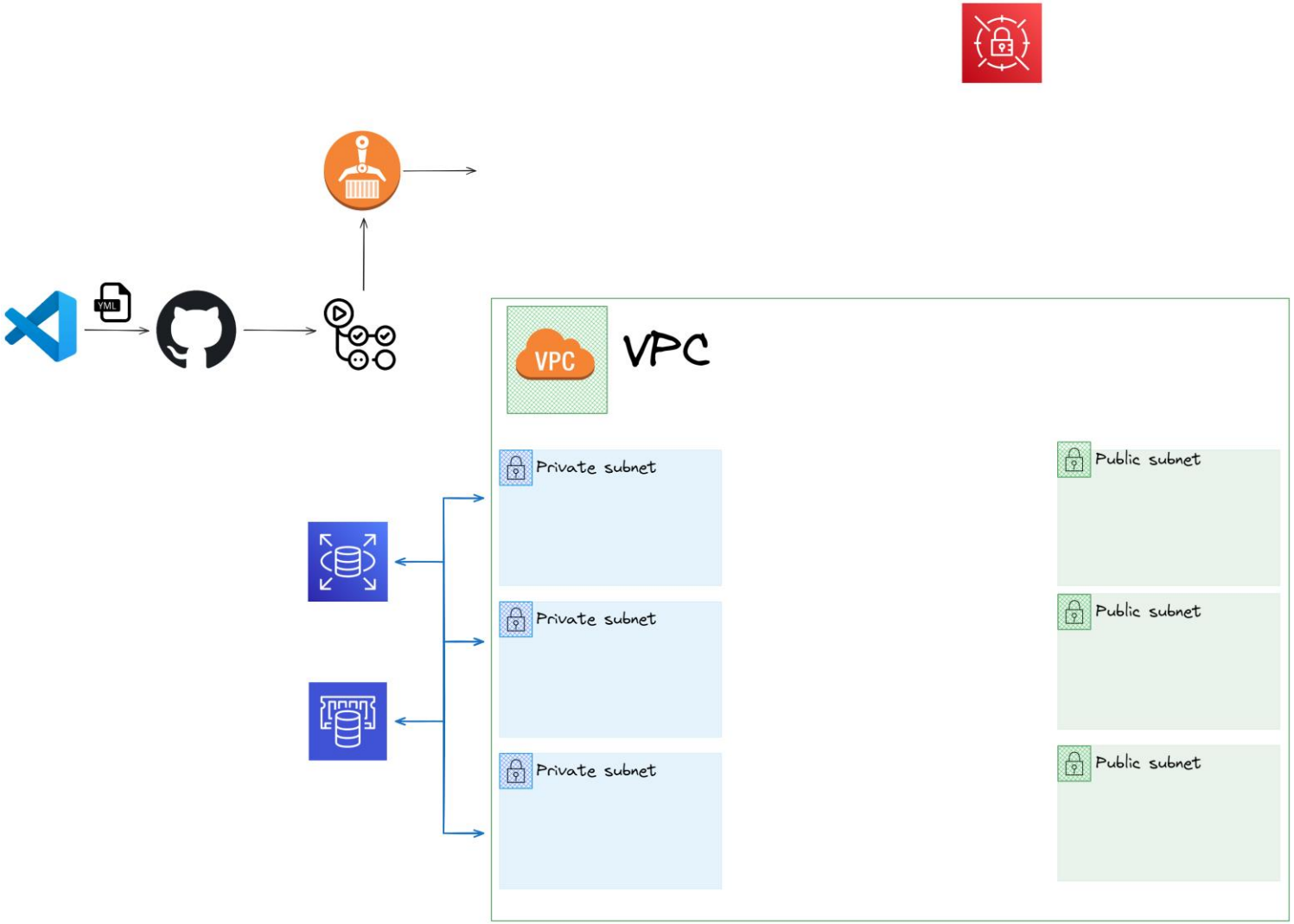
엔진	엔진 버전	포트
Redis	7.1	6379
파라미터 그룹	**노드 유형**	**복제본 개수**
default.redis7	cache.r7g.large	0

연결
이 클러스터가 지원할 IP 버전을 선택합니다. 그런 다음 기존 서브넷 그룹을 선택하거나 새 서브넷 그룹을 생성합니다.

네트워크 유형	서브넷 그룹	가용 영역
IPv4	jbnu-shopping-redis	ap-northeast-2a, ap-northeast-2b, ap-northeast-2c

Part 6

AWS ElastiCache



PART 7.

AWS ECS

AWS ECS

001 >> ECS Cluster

ECR에 올라온 태스크들이 올라갈 ECS 클러스터를 생성합니다.

002 >> Task 정의

Frontend, Backend Application가 Cluster에 올라갈 컨테이너 실행 환경 Task를 정의합니다.

003 >> Service 등록

Task를 기준으로 삼아 작동할 Service를 등록합니다. Service를 등록하면서 함께 ELB그룹을 설정합니다. 만일 배포 방식을 Green-Blue 방식으로 도입하면 CodeDepoly도 함께 생성이 됩니다.
각 컨테이너는 EC2 방식이 아닌 Fargate 방식으로 배포가 됩니다.

Part 7

AWS ECS-Cluster

클러스터 jbnu-shopping-cluster이(가) 생성되었습니다. 클러스터 보기

jbnu-shopping-cluster

마지막 업데이트: 2024년 12월 20일 14:45 (UTC+9:00) 클러스터 업데이트 클러스터 삭제

클러스터 개요

ARN	상태	CloudWatch 모니터링	등록된 컨테이너 인스턴스
[redacted]	활성	기본값	-

서비스

드레이닝 중	활성
-	-

태스크

대기 중	실행 중
-	-

암호화

관리형 스토리지	Fargate 임시 스토리지
-	-

서비스 | 태스크 | 인프라 | 지표 | 예약된 태스크 | 태그

서비스 (0) 정보

태그 관리 업데이트 서비스 삭제 생성

값으로 서비스 필터링

시작 유형 필터링: 모든 시작 유형

서비스 유형 필터링: 모든 서비스 유형

1

Service name	ARN	Status	서비스 유형	Deployments and tasks	Last deployment	Task definition	Launch type

서비스 없음

표시할 서비스가 없습니다.

생성

Part 7

AWS ECS-Task JSON

태스크 정의 구성

태스크 정의 패밀리 | 정보

고유한 태스크 정의 패밀리 이름을 지정합니다.

jbnu-shopping-backend-task

최대 255개의 문자(대문자 및 소문자), 숫자, 하이픈 및 밑줄이 허용됩니다.

▼ 인프라 요구 사항

태스크 정의에 대한 인프라 요구 사항을 지정합니다.

시작 유형 | 정보

시작 유형을 선택하면 작업 정의 파라미터가 변경됩니다.

☑ AWS Fargate

컨테이너용 서버리스 컴퓨팅입니다.

☐ Amazon EC2 인스턴스

Amazon EC2 인스턴스를 사용하는 자체 관리형 인프라입니다.

OS, 아키텍처, 네트워크 모드

네트워크 모드는 작업에 사용되며 선택한 컴퓨팅 유형에 따라 달라집니다.

운영 체제/아키텍처 | 정보

Linux/X86_64

네트워크 모드 | 정보

awsvpc

태스크 크기 | 정보

태스크를 위해 예약할 CPU 및 메모리의 양을 지정합니다.

CPU

1 vCPU

메모리

3 GB

▼ 태스크 역할 - *조건부*

태스크 역할 | 정보

태스크 IAM 역할은 태스크의 컨테이너가 AWS 서비스에 API 요청을 할 수 있도록 허용합니다. IAM 콘솔 에서 태스크 IAM 역할을 생성할 수 있습니다.

jbnu-shopping-backend-task-iam

Part 7

AWS ECS-Service

상태 정보

서비스 이름
jbnu-shopping-frontend-ecs

서비스 ARN
arn:aws:ecs:ap-northeast-2:116981803095:service/jbnu-shopping-cluster/jbnu-shopping-frontend-ecs

배포 현재 상태
1개의 완료된 작업

생성 시간
2024년 12월 22일 09:32 (UTC+9:00)

상태 검사 유예 기간
0초

[redacted]

▼ 로드 밸런서 상태

(Application Load Balancer) elb-ecs-frontend

로드 밸런서 보기

리스너 프로토콜:포트
HTTP:80

대상 그룹 이름:프로토콜
ecs-frontend:HTTP

상태 확인 경로
/

목표 (합계 1)
1 정상 0 비정상

상태 정보

서비스 이름
ecs-backend-1

서비스 ARN
[redacted]

배포 상태
성공

생성 시간
2024년 12월 22일 21:03 (UTC+9:00)

상태 검사 유예 기간
0초

▼ 로드 밸런서 상태

(Application Load Balancer) elb-ecs-backend

로드 밸런서 보기

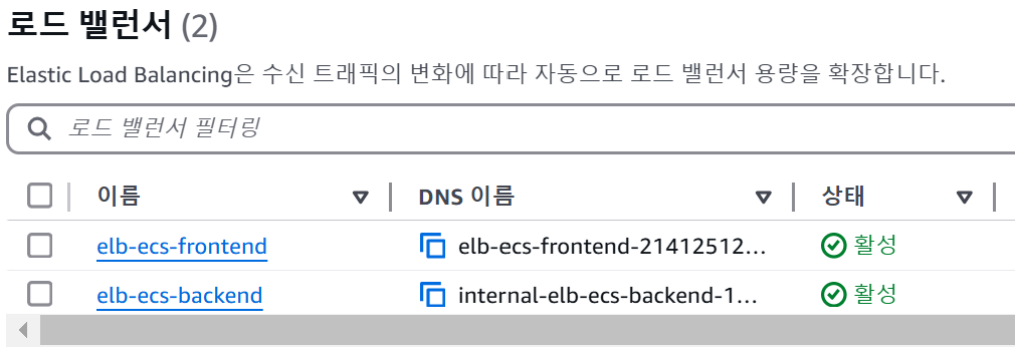
리스너 프로토콜:포트
HTTP:80

대상 그룹 이름:프로토콜
g2-ecs-backend:HTTP

상태 확인 경로
/swagger-ui/index.html

목표 (합계 1)
1 정상 0 비정상

AWS ECS-ELB



001 >> Frontend ELB

Frontend ELB의 경우 Public VPC에 위치하며, 남들이 쉽게 접근 할 수 있도록 해야 하기에 인터넷 경계에 위치하는 ELB로 구성하였습니다.

002 >> Backend ELB

Backend ELB의 경우 Private VPC에 위치하며, 내부 IP주소 라우팅 용으로 사용하기 때문에 내부에 위치하는 ELB로 구성하였습니다.

Part 7

AWS ECS

Service로 올라간 Frontend Project ELB 주소로 접근 성공

AWS ECS

회원가입

아이디

test1test1

비밀번호

••••••••••

비밀번호를 8자~15자 이내로 입력해주세요

비밀번호 확인

••••••••••

비밀번호를 8자~15자 이내로 입력해주세요

이름

test1test1

Always match Chrome's language | Switch DevTools to Korean | Don't show again

Elements | Console | Sources | Network » 1 59 13

top ▼ | Filter | Default levels ▼ | 14 Issues: 13 1 | 2 hidden

▶ 59 Chrome is moving towards a new experience that allows users to choose to browse without third-party cookies.

▶ POST page-7c7cd8b319e03aa7.js:1
http://elb-ecs-frontend-2141251244.ap-northeast-2.elb.amazonaws.com/api/sign-up 400 (Bad Request)

page-7c7cd8b319e03aa7.js:1

```
▼ {timestamp: '2024-12-23T13:49:08.137942567', status: 400, message: '회원 가입 요청 값이 올바르지 않습니다. 다시 확인해 주세요.', data: Array(2)}
  ▶ data: (2) [{…}, {…}]
    message: "회원 가입 요청 값이 올바르지 않습니다. 다시 확인해 주세요."
    status: 400
    timestamp: "2024-12-23T13:49:08.137942567"
  ▶ [[Prototype]]: Object
```

page-7c7cd8b319e03aa7.js:1

```
▼ {timestamp: '2024-12-23T13:49:19.826176583', status: 201, message: '회원가입이 성공적으로 완료되었습니다.', data: null}
    data: null
    message: "회원가입이 성공적으로 완료되었습니다."
    status: 201
    timestamp: "2024-12-23T13:49:19.826176583"
  ▶ [[Prototype]]: Object
```

Service로 올라간 두 컨테이너간 정상적인 연결도 성공

AWS ECS

개발자 도구 > CodeDeploy > 애플리케이션

애플리케이션

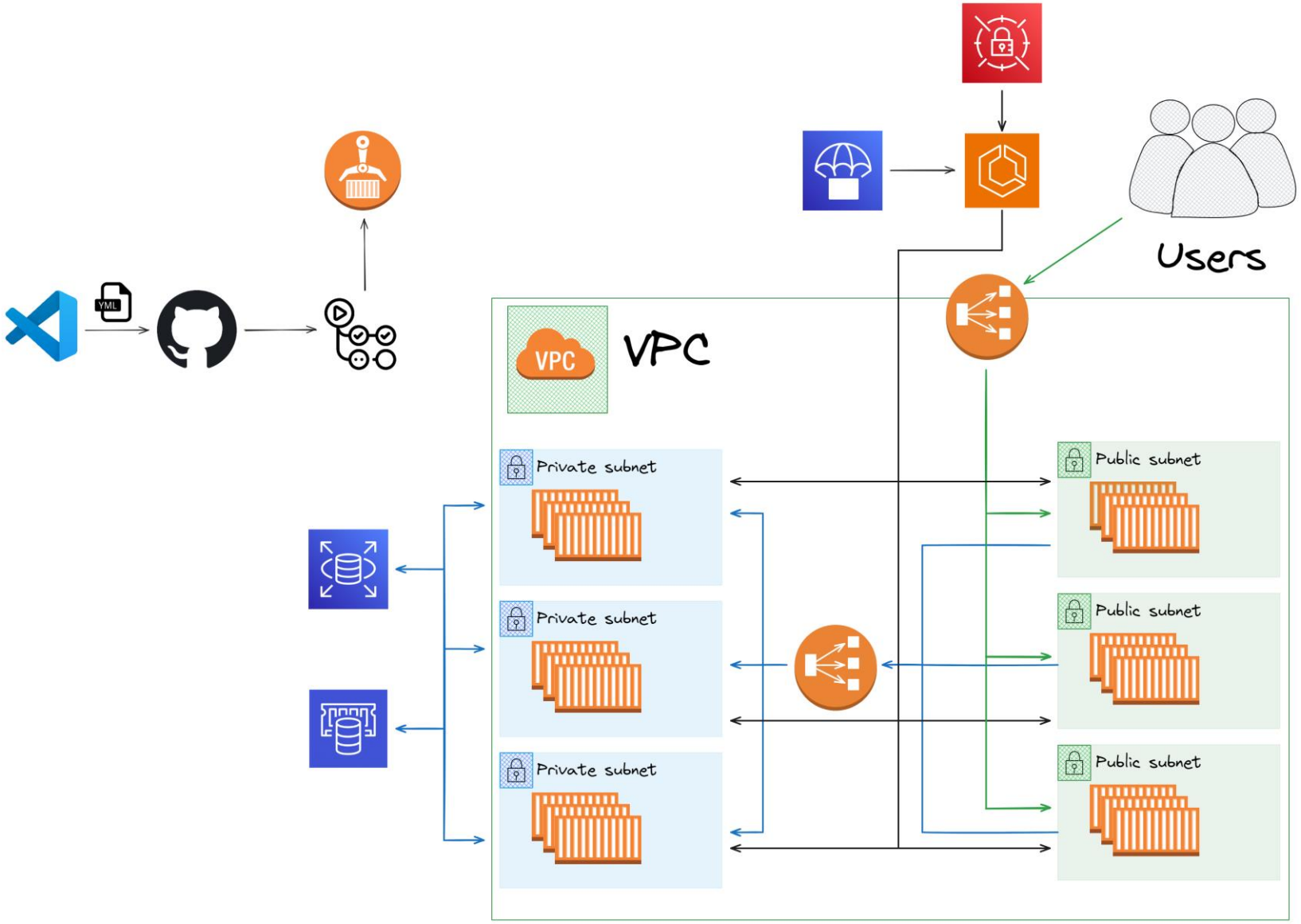
알림 | 세부 정보 보기 | 애플리케이션 배포 | 애플리케이션 생성

1

	애플리케이션 이름	컴퓨팅 플랫폼	생성됨
○	AppECS-jbnu-shopping-cluster-ec...	Amazon ECS	1일 전
○	AppECS-jbnu-shopping-cluster-ec...	Amazon ECS	1일 전

Part 7

AWS ECS



PART 8.

CI/CD

CI/CD

001 >> EventBridge 작성

ECR 레포지토리에 이미지가 올라가면 트리거 발동

002 >> Lambda 함수 작성

발동된 트리거를 대상으로 기존 Task 정의를 갖고 새 Task 정의를 만들고, ECS Clinent 에게 신호를 보내 CodeDepoly에 배포하도록 실행하는 Lambda 함수 작성

CI/CD

ECRImagePushBackend

규칙 세부 정보 정보

규칙 이름
ECRImagePushBackend

설명

상태
활성화됨

규칙 ARN
arn:aws:even
1803095:rule/ECI

이벤트 패턴 | 대상 | 모니터링 | 태그

이벤트 패턴 정보

```
1 {
2   "source": ["aws.ecr"],
3   "detail-type": ["ECR Image Action"],
4   "detail": {
5     "action-type": ["PUSH"],
6     "repository-name": ["jbnu-shopping/backend/image"],
7     "image-tag": ["latest"]
8   }
9 }
```

ECRImagePushFrontend

규칙 세부 정보 정보

규칙 이름
ECRImagePushFrontend

설명

상태
활성화됨

규칙 ARN
arn:aws:eve
1803095:rule/E

이벤트 패턴 | 대상 | 모니터링 | 태그

이벤트 패턴 정보

```
1 {
2   "source": ["aws.ecr"],
3   "detail-type": ["ECR Image Action"],
4   "detail": {
5     "action-type": ["PUSH"],
6     "repository-name": ["jbnu-shopping/frontend/image"],
7     "image-tag": ["latest"]
8   }
9 }
```

CI/CD λ

lambda_function.py

```python
def lambda_handler(event, context):
    try:

        # taskRoleArn을 제외할지 조건부로 설정
        task_role_arn = task_definition.get('taskRoleArn', None)

        # 새 태스크 정의 등록
        register_params = {
            "family": task_definition['family'],
            "containerDefinitions": container_definitions,
            "networkMode": task_definition['networkMode'],
            "cpu": task_definition['cpu'],
            "memory": task_definition['memory'],
            "executionRoleArn": task_definition['executionRoleArn'],
            "requiresCompatibilities": task_definition['requiresCompatibilities']
        }

        # taskRoleArn이 None이 아닌 경우에만 추가
        if task_role_arn:
            register_params["taskRoleArn"] = task_role_arn

        new_task_definition = ecs_client.register_task_definition(**register_params)
        new_task_arn = new_task_definition['taskDefinition']['taskDefinitionArn']
        print(f"새 태스크 정의 등록됨: {new_task_arn}")

# CodeDeploy 배포 생성
        deployment_response = codedeploy_client.create_deployment(
            applicationName=CODEDEPLOY_APP_NAME,
            deploymentGroupName=CODEDEPLOY_DEPLOYMENT_GROUP,
            revision={
                'revisionType': 'AppSpecContent',
```

Ln 46, Col 23 Spaces: 4 UTF-8 LF Python Lambda Layout: US

Part 8

CI/CD

aws.yaml

on: push

build-and-push-frontend 3s

Deploying to **test**

build-and-push-backend 3s

Deploying to **test**

시작 유형 필터링

모든 시작 유형

서비스 유형 필터링

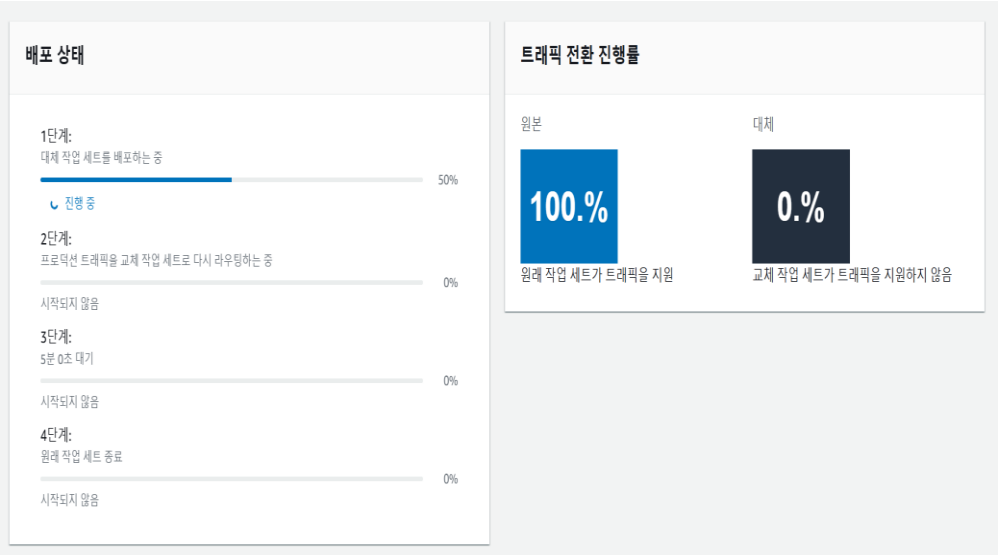
모든 서비스 유형

	서비스 유형	Deployments and tasks	Last deployment
	REPLICA	1/1 태스크 실행 중	진행 중
	REPLICA	1/1 태스크 실행 중	진행 중

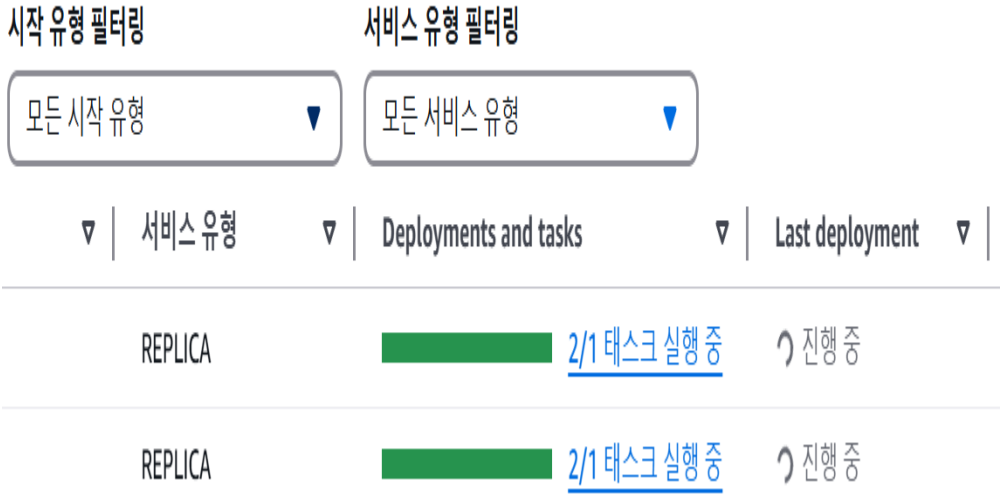
	배포 ID	상태	배포 유형
○	d-GJGD8FPA9	진행 중	블루/그린
○	d-0R9HRKPA9	진행 중	블루/그린

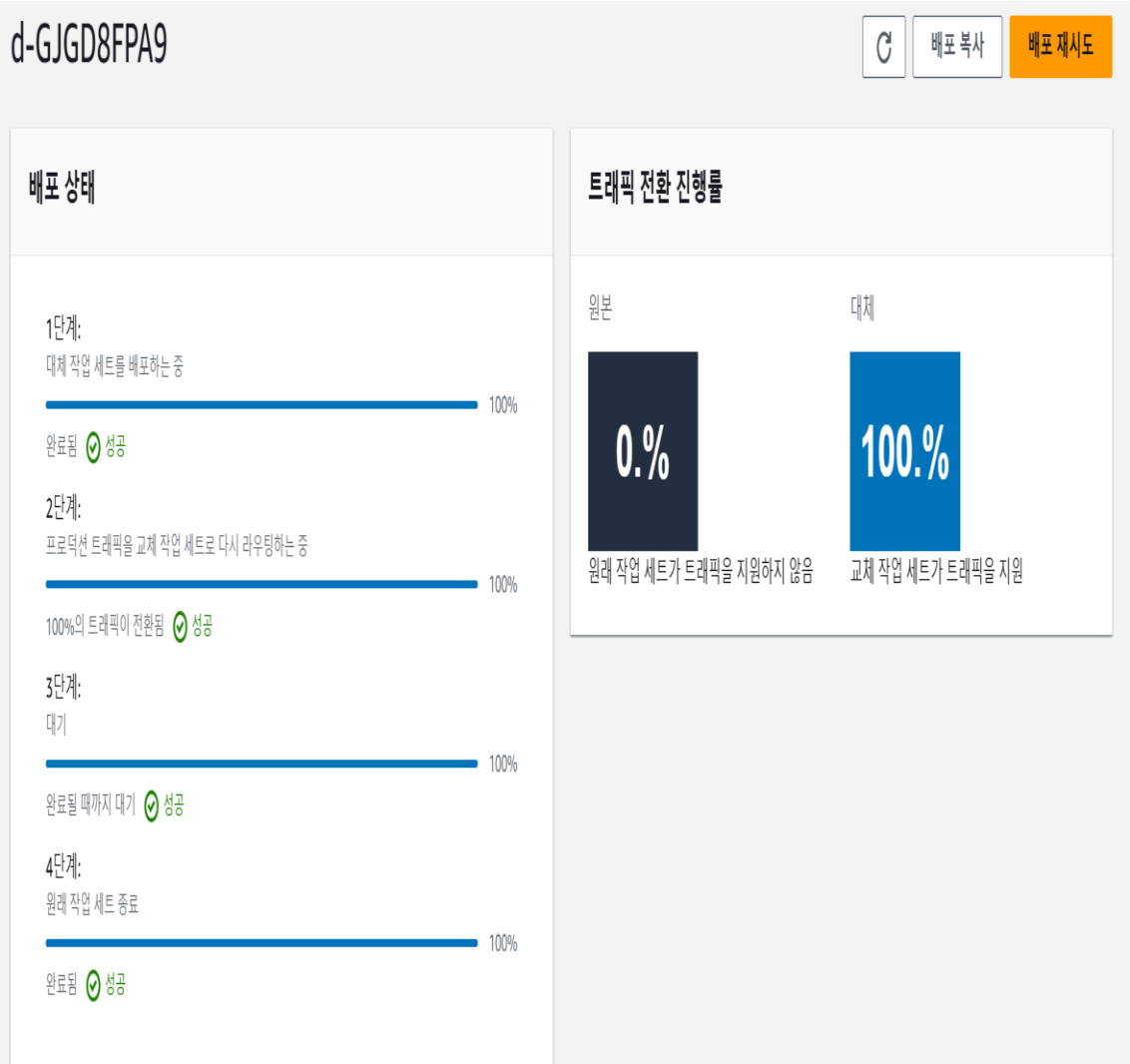
Part 8

CI/CD


배포 상태
1단계:
대체 작업 세트를 배포하는 중
50%
진행 중
2단계:
프로덕션 트래픽을 교체 작업 세트로 다시 라우팅하는 중
0%
시작되지 않음
3단계:
5분 0초 대기
0%
시작되지 않음
4단계:
원래 작업 세트 종료
0%
시작되지 않음
트래픽 전환 진행률
원본
100.%
원래 작업 세트가 트래픽을 지원
대체
0.%
교체 작업 세트가 트래픽을 지원하지 않음

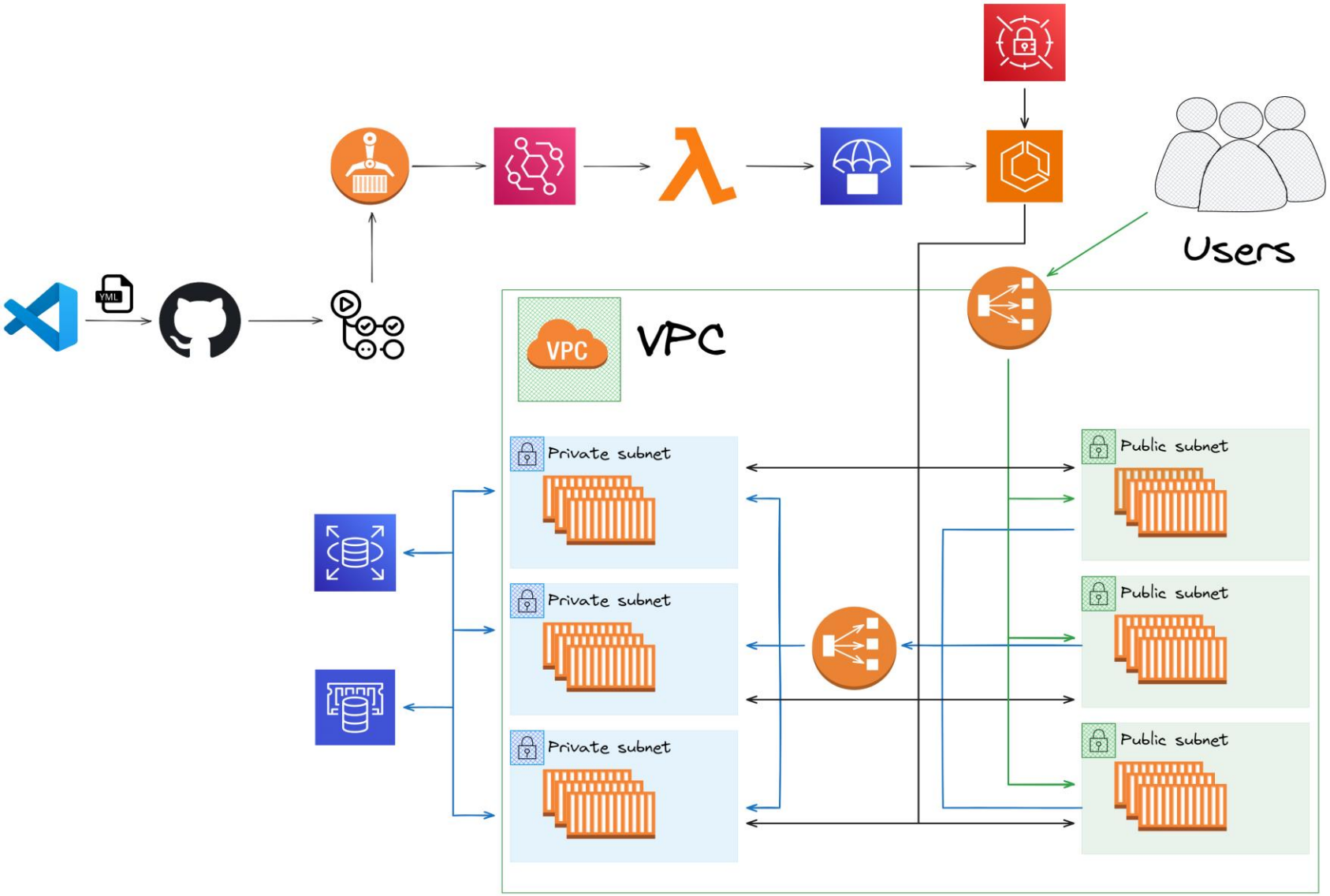
태그 관리


시작 유형 필터링
모든 시작 유형
서비스 유형 필터링
모든 서비스 유형
서비스 유형
Deployments and tasks
Last deployment
REPLICA
2/1 태스크 실행 중
진행 중
REPLICA
2/1 태스크 실행 중
진행 중

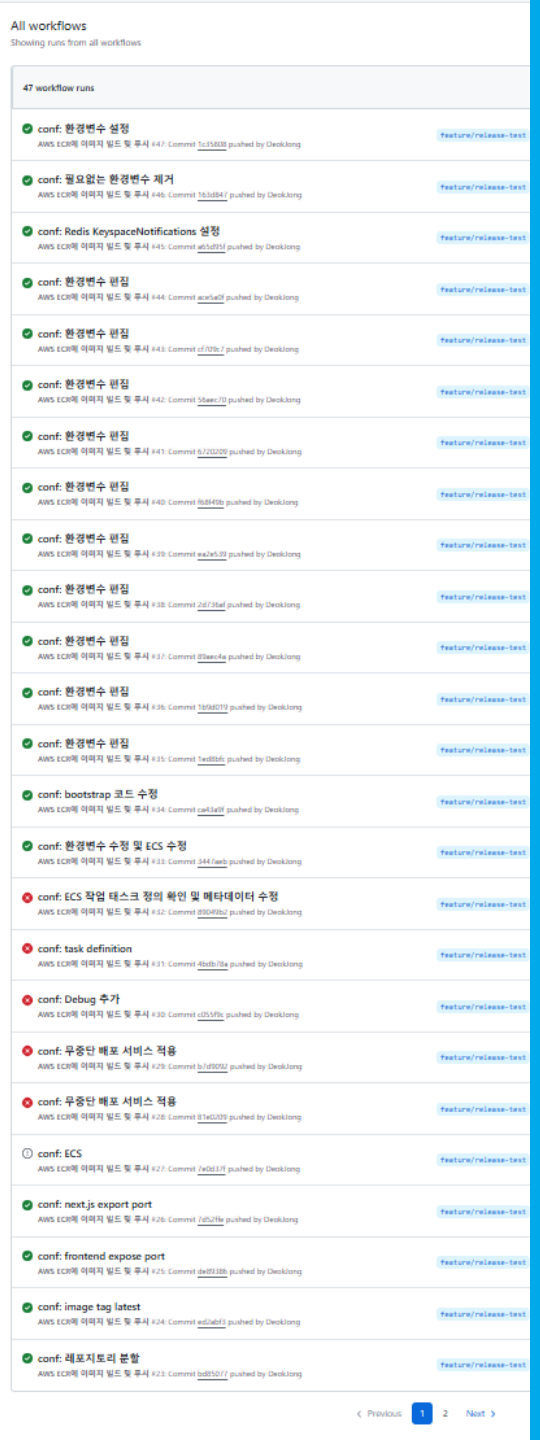

d-GJGD8FPA9
배포 복사
배포 재시도
배포 상태
1단계:
대체 작업 세트를 배포하는 중
100%
완료됨 성공
2단계:
프로덕션 트래픽을 교체 작업 세트로 다시 라우팅하는 중
100%
100%의 트래픽이 전환됨 성공
3단계:
대기
100%
완료될 때까지 대기 성공
4단계:
원래 작업 세트 종료
100%
완료됨 성공
트래픽 전환 진행률
원본
0.%
원래 작업 세트가 트래픽을 지원하지 않음
대체
100.%
교체 작업 세트가 트래픽을 지원

Part 8

CI/CD


YML
Users
VPC
VPC
Private subnet
Private subnet
Private subnet
Public subnet
Public subnet
Public subnet

어려웠던점?



- 설명으로는 매우 간단하게 보이나, 실제로는 상당히 많은 시도를 하였습니다.
- 먼저 Git Actions 와 관련해서 한 Actions에서 병렬처리 하여 두 ECR에 이미지를 올리는데 상당히 애먹었습니다.

로그 스트림 (53)

로그 스트림 필터링 또는 접두사 검색 시도

정확히 일

로그 스트림	마지막 이벤트 시간
ecs/jbnu-shopping-backend-container/0473ece4de354dba8b223d92602f1	2024-12-23 13:33:17 (UTC)

백엔드와 관련해서는 많은 문제가 있었습니다.

1. ELB의 보안그룹과 ECS Task의 보안그룹을 서로 같게하여 Spring boot의 경우 엄격한 보안그룹 설정으로 인해 ELB의 Health checks가 계속 실패하였습니다.
   이에 대한 해결은 Task의 보안그룹은 내부 ELB의 접속 허용이며, ELB의 형태를 기존 Task의 보안그룹과 같게하니 되는것 처럼 보였습니다.
2. 그러나, 계속 Health checks가 뜨는 것을 보고, 상태경로가 잘못됐다는 것을 깨달았습니다. Spring boot는 기본 루트 도메인에선 404에러를 보내 Health Checks가 실패하였습니다. 이후 경로를 스웨거 경로로 바꿔서 정상적으로 실행되는 것을 보았습니다.

감사합니다